

untangle::async

Generated by Doxygen 1.18.0

1	async	1
2	Topic Index	5
2.1	Topics	5
3	Namespace Index	7
3.1	Namespace List	7
4	Class Index	9
4.1	Class List	9
5	File Index	11
5.1	File List	11
6	Topic Documentation	13
6.1	namespace untangle: functions	13
7	Namespace Documentation	15
7.1	untangle Namespace Reference	15
7.1.1	Detailed Description	15
8	Class Documentation	17
8.1	untangle::async::attachment Struct Reference	17
8.1.1	Detailed Description	17
8.2	untangle::async::execution< actionT > Class Template Reference	17
8.2.1	Detailed Description	18
8.2.2	Constructor & Destructor Documentation	19
8.2.2.1	execution() [1/2]	19
8.2.2.2	execution() [2/2]	19
8.2.2.3	~execution()	19
8.2.3	Member Function Documentation	20
8.2.3.1	create_instance()	20
8.2.3.2	bind_action_and_method()	20
8.2.3.3	bind_action_and_function()	21
8.2.3.4	is_running()	21
8.2.3.5	is_busy()	22
8.2.3.6	stop()	22
8.2.3.7	add_action()	22
8.2.3.8	attach()	23
8.2.3.9	detach()	24
8.2.3.10	results()	24
8.3	untangle::async::execution_poll Class Reference	25
8.3.1	Detailed Description	25
8.3.2	Member Function Documentation	25
8.3.2.1	add()	25
8.3.2.2	remove()	26

8.3.2.3 <code>is_running()</code>	26
8.3.2.4 <code>get()</code>	26
8.4 <code>untangle::async::invalid_attachment</code> Struct Reference	27
8.4.1 Detailed Description	27
8.4.2 Constructor & Destructor Documentation	27
8.4.2.1 <code>invalid_attachment()</code>	27
8.4.3 Member Function Documentation	27
8.4.3.1 <code>what()</code>	27
9 File Documentation	29
9.1 <code>async.hpp</code>	29
Index	35

Chapter 1

async

c++ mechanism to run queued callables in an asynchronous fashion

The interface consists of one header file that exposes a generic execution class. It is a wrapper of a list of callables that are invoked on other thread. The callables are pumped into the list by invoking an action from the caller thread.

The implementation is using the **actuator** callable, and therefore it exemplifies how a generic callable may improve the code structure, by easily creating interfaces inside a class instead of using external ones.

There are provided two execution modes: one-off and continuous.

requirements

C++23. The interface reports warnings with `std::println`, so it needs a standard library that provides `<print>` — GCC 14 or newer; older `libc++` and `MSVC` releases may not have it yet.

one-off execution



example: one-off execution of a plain function

```
#include <async.hpp>

void f()
{
    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type; it must be held by a shared pointer,
    // because an async binding keeps only weak ownership of it
    // the name is optional, and identifies the execution in the warnings it reports
    using void_execution = untangle::async::execution<std::function<void(void)>>;
    auto execution = void_execution::create_instance("oneoff");
```

```

std::function<void(void)> action;
// create async binding between the action and f; the execution is passed as a shared pointer,
// which the binding holds weakly - that is what lets the action outlive it safely
void_execution::bind_action_and_function(action, f, execution);
// whenever the action is invoked, a new callable wrapping f will be added to the execution's action list
action();
// run the execution
execution->run();

// add the execution object to the running poll
untangle::async::execution_poll::get().add(*execution);

// wait the polled executions to finish
while(untangle::async::execution_poll::get().is_running())
{
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
}

return 0;
}

```

example: one-off execution of a class method

```

#include <async.hpp>

class A
{
public:
    void f(int x)
    {
        std::cout << "A::f " << x << " thread " << std::this_thread::get_id() << std::endl;
    }
    std::function<void(int)> action;
};

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    auto a = std::make_shared<A>();

    // declare an execution specialized for an action type; it must be held by a shared pointer
    using int_execution = untangle::async::execution<std::function<void(int)>>;
    auto execution = int_execution::create_instance("oneoff_method");
    // note both the bound object and the execution must be shared pointers
    int_execution::bind_action_and_method(a->action, a, &A::f, execution);
    // whenever the action is invoked, a new callable wrapping f will be added to the execution's action list
    a->action(10);
    // run the execution
    execution->run();

    // add the execution object to the running poll
    untangle::async::execution_poll::get().add(*execution);

    // wait the polled executions to finish
    while(untangle::async::execution_poll::get().is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}

```

continuous execution



example: continuous execution of a plain function

```

#include <async.hpp>

void f()
{
    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

```

```

}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type; it must be held by a shared pointer
    using void_execution = untangle::async::execution<std::function<void(void)>>;
    auto execution = void_execution::create_instance("continuous");
    std::function<void(void)> action;
    // create async binding between the action and f
    void_execution::bind_action_and_function(action, f, execution);

    // run the execution
    execution->start();
    // add f to the execution's queue (three times)
    action();action();action();
    // stop the execution
    execution->stop();

    // add the execution object to the running poll
    untangle::async::execution_poll::get().add(*execution);

    // wait the polled executions to finish
    while(untangle::async::execution_poll::get().is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}

```

queueing and lifetime

Queueing is thread safe. An action may be invoked from any thread; the execution's list is guarded, and the worker is woken rather than polled for.

`stop()` ends the execution's working life. The worker runs whatever was queued before the call and then leaves. An action invoked after `stop()` is refused rather than queued, because it would never run, and the refusal is reported:

```
warning: execution 'continuous' is stopped, action not added
```

An action bound to an object that has since been destroyed is dropped the same way, rather than ending the process:

```
warning: execution 'continuous' dropped an invalid action: bind: invalid object
```

The worker is detached and cannot be joined. Waiting for an execution therefore means waiting on its running state, which is what `execution_poll` reports and what the examples above do. The destructor waits on the same state, so an execution that goes out of scope while its worker is still running blocks until the worker is finished rather than leaving it reading freed memory.

The poll holds a pointer to each execution added to it, so an execution withdraws itself in its destructor. `execution_poll::remove()` is available for withdrawing one earlier; calling it is not required.

building the tests

```

cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build

```

`test/CMakeLists.txt` fetches googletest at configure time, so the first configure needs network access. The `actuator` submodule has to be present: `git submodule update --init`.

sanitizers

ASYNC_SANITIZE builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```
cmake -S test -B test/build-asan -DASYNC_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan
```

Accepted values are address, thread, undefined, or empty. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. address and thread instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

The two answer different questions. ThreadSanitizer names both sides of a data race, which is how the queue's races were diagnosed. AddressSanitizer reports a use-after-free, which is how the dangling attachment was. Neither sees everything: a defect reached through a destroyed `std::mutex` throws `std::system_error` from inside `libsystem` before any instrumented load runs, so both sanitizers stay silent. Reach for a direct instrumented read when that happens.

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

namespace untangle: functions [13](#)

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	Interface to untangle::async::execution class	15
--------------------------	---	--------------------

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::async::attachment	
One execution's place in the attachment graph: who attached it, if anyone	17
untangle::async::execution< actionT >	
Asynchronous template execution class	17
untangle::async::execution_poll	
Execution poll class	25
untangle::async::invalid_attachment	
Invalid attachment exception	27

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

async.hpp		29
---------------------------	-------	----

Chapter 6

Topic Documentation

6.1 namespace untangle: functions

Chapter 7

Namespace Documentation

7.1 untangle Namespace Reference

Interface to [untangle::async::execution](#) class.

7.1.1 Detailed Description

Interface to [untangle::async::execution](#) class.

Chapter 8

Class Documentation

8.1 `untangle::async::attachment` Struct Reference

One execution's place in the attachment graph: who attached it, if anyone.

```
#include <async.hpp>
```

Public Attributes

- `std::weak_ptr< attachment >` `attacher`
The link of the execution that attached this one; empty when nothing has.

8.1.1 Detailed Description

One execution's place in the attachment graph: who attached it, if anyone.

Deliberately at namespace scope rather than nested in [execution](#), so that it is one type rather than one per specialisation. An execution may be attached by an execution of any specialisation, and the whole point of this record is that the chain can be walked without knowing what any link's action type is.

An execution holds one of these by `std::shared_ptr` and it dies with the execution, so it doubles as the liveness token the attached side needs: a link whose weak reference still locks belongs to an execution that is still there.

The documentation for this struct was generated from the following file:

- `async.hpp`

8.2 `untangle::async::execution< actionT >` Class Template Reference

Asynchronous template execution class.

```
#include <async.hpp>
```

Public Member Functions

- [execution](#) ()
Constructs a new execution object with the default name. Call [create_instance\(\)](#) instead.
- [execution](#) (std::string exec_name)
Constructs a new named execution object.
- [~execution](#) ()
Destroys the execution object, once its worker has left.
- bool [is_running](#) () const
Checks if this execution has finished.
- bool [is_busy](#) () const
Is this execution working through its action list?
- void [stop](#) ()
Stops the worker, after the actions already queued have run.
- template<typename... Args>
void [add_action](#) (actionT action, Args... args)
- template<typename otherT>
void [attach](#) (const std::shared_ptr< otherT > &other)
Attaches another execution, so that this one triggers it.
- template<typename otherT>
bool [detach](#) (otherT &other)
Detaches an execution previously attached by [attach\(\)](#).
- auto [results](#) () const
The return values of the actions run by [run\(\)](#), in the order they ran.

Static Public Member Functions

- template<typename... Args>
static std::shared_ptr< execution > [create_instance](#) (Args &&... args)
Creates an execution owned by a std::shared_ptr, which is what binding to it requires.
- template<typename classT, typename T>
static void [bind_action_and_method](#) (actionT &action, const std::shared_ptr< classT > &obj, T classT::*method, const std::shared_ptr< execution > &async_exec)
Binds asynchronously an external action to a class function member.
- template<typename T>
static void [bind_action_and_function](#) (actionT &action, const T &Fn, const std::shared_ptr< execution > &async_exec)
Binds asynchronously an external action to a plain function.

Static Public Attributes

- static constexpr auto default_name = "default_name"
Name of an execution built unnamed.

8.2.1 Detailed Description

```
template<typename actionT>
class untangle::async::execution< actionT >
```

Asynchronous template execution class.

Template Parameters

actionT	It represents the type of the action. It is specified as <code>std::function<...></code> and should match the signature of the bound function or class method.
---------	--

8.2.2 Constructor & Destructor Documentation

8.2.2.1 execution() [1/2]

```
template<typename actionT>
untangle::async::execution< actionT >::execution () [inline]
```

Constructs a new execution object with the default name. Call [create_instance\(\)](#) instead.

The name identifies an execution in the warnings it reports, and is otherwise unused. An execution built without one keeps the default name rather than an empty one, so a warning always names something.

8.2.2.2 execution() [2/2]

```
template<typename actionT>
untangle::async::execution< actionT >::execution (
    std::string exec_name) [inline], [explicit]
```

Constructs a new named execution object.

An execution constructed directly cannot be bound to - [bind_action_and_method\(\)](#) and [bind_action_and_function\(\)](#) require a `std::shared_ptr`. Use [create_instance\(\)](#) for one that is going to carry bound actions.

Parameters

exec_name	- A name for this execution.
-----------	------------------------------

8.2.2.3 ~execution()

```
template<typename actionT>
untangle::async::execution< actionT >::~~execution () [inline]
```

Destroys the execution object, once its worker has left.

The worker is detached and cannot be joined, so this waits on the same "running" state that [execution_poll](#) reports - the only handle a detached worker offers. It returns at once for an execution that was never started, or that the caller has already polled to a stop.

Remarks

It first takes this execution out of whatever attached it, which is the inverse of [attach\(\)](#) and the counterpart of the [execution_poll::remove\(\)](#) below.

This is safe only because `running_` is cleared as the very last thing the worker does. Nothing may be added after it in `execute()` or `loop()`: the object can be freed the moment it reads false.

8.2.3 Member Function Documentation

8.2.3.1 create_instance()

```
template<typename actionT>
template<typename... Args>
std::shared_ptr< execution > untangle::async::execution< actionT >::create_instance (
    Args &&... args) [inline], [static]
```

Creates an execution owned by a `std::shared_ptr`, which is what binding to it requires.

`bind_action_and_method()` and `bind_action_and_function()` take the execution as a `std::shared_ptr` and hold it weakly, so an execution that is to be bound has to be created this way. One built on the stack, as a data member, or through a bare new is still a usable execution

- actions can be handed to `add_action()` directly - it simply cannot be bound, and the attempt does not compile.

The arguments are forwarded to a constructor, so this factory does not have to be revisited when one is added.

Parameters

args	- Constructor arguments: a name, or nothing for the default name.
------	---

Returns

- A `std::shared_ptr` owning the new execution.

8.2.3.2 bind_action_and_method()

```
template<typename actionT>
template<typename classT, typename T>
void untangle::async::execution< actionT >::bind_action_and_method (
    actionT & action,
    const std::shared_ptr< classT > & obj,
    T classT::* method,
    const std::shared_ptr< execution< actionT > > & async_exec) [inline], [static]
```

Binds asynchronously an external action to a class function member.

It creates an action as a binding to a class method (by `untangle::bind()`), and assigns to action a callable that passes it to `add_action()`. The execution is taken as a `std::shared_ptr` and held as a `std::weak_ptr`, exactly as `untangle::bind()` holds the bound object: the action can therefore outlive the execution and report a dead binding rather than following a dangling reference.

Attention

Invoking action after the execution has been destroyed throws `untangle::invalid_action`. `execute↔actions()` catches it, so such an action is dropped with a warning rather than ending the worker thread.

Parameters

action	[in,out] - An action of type <code>std::function<...></code> .
obj	- A <code>std::shared_ptr</code> that wraps the bound class object.
method	- Pointer to function member. It is specified as <code>&<class type>::<function member></code> .
async_exec	- A <code>std::shared_ptr</code> owning the execution the action is queued on. Requiring it here is what keeps an execution that no <code>shared_ptr</code> owns from being bound at all.

8.2.3.3 bind_action_and_function()

```
template<typename actionT>
template<typename T>
void untangle::async::execution< actionT >::bind_action_and_function (
    actionT & action,
    const T & Fn,
    const std::shared_ptr< execution< actionT > > & async_exec) [inline], [static]
```

Binds asynchronously an external action to a plain function.

It wraps `Fn` in an action and assigns to `action` a callable that passes it to `add_action()`. The execution is held weakly, for the reason given on `bind_action_and_method()`.

Parameters

action	[in,out] - An action of type <code>std::function<...></code> .
Fn	- A plain function.
async_exec	- A <code>std::shared_ptr</code> owning the execution the action is queued on.

8.2.3.4 is_running()

```
template<typename actionT>
bool untangle::async::execution< actionT >::is_running () const [inline]
```

Checks if this execution has finished.

Returns

true - The execution has not finished.
false - The execution has finished.

Is this execution still working?

False from the moment the worker starts reporting itself finished, which is before `running_` is cleared - the two answer different questions. This one is for callers, and says whether there is still work going on; `running_` is the handshake `~execution()` waits on, and says whether the worker is still touching this object. They were the same answer until the notification needed to be told the truth about itself.

8.2.3.5 `is_busy()`

```
template<typename actionT>
bool untangle::async::execution< actionT >::is_busy () const [inline]
```

Is this execution working through its action list?

True while anything is queued, and while an action taken off the queue is still running - the gap between those two is why this is not simply a test for an empty list. An execution is busy from the moment an action is added until the last one has returned.

Distinct from [is_running\(\)](#), which is about the worker thread. A continuous worker started by `start()` is running for its whole life whether or not it has anything to do, so [is_running\(\)](#) cannot answer "has this one got room for more work"; this can. A one-shot `run()` is both.

Remarks

Answers for this execution's own actions. An execution that is triggering others through [attach\(\)](#) reports on its own list, not on theirs - see item 12.

A true answer is a fact about the instant it was taken, and the execution may drain the moment after. A false answer is durable only if the caller is the one adding the actions, which is the case this is written for: asking whether a worker has room before handing it more.

Returns

true - actions are queued, or one is running.

8.2.3.6 `stop()`

```
template<typename actionT>
void untangle::async::execution< actionT >::stop () [inline]
```

Stops the worker, after the actions already queued have run.

Remarks

An action passed to [add_action\(\)](#) after this call is refused, not queued: [stop\(\)](#) is the end of this execution's working life, and an action accepted then would never run.

8.2.3.7 `add_action()`

```
template<typename actionT>
template<typename... Args>
void untangle::async::execution< actionT >::add_action (
    actionT action,
    Args... args) [inline]
```

Template Parameters

Args	
------	--

Parameters

action	
args	

8.2.3.8 attach()

```
template<typename actionT>
template<typename otherT>
void untangle::async::execution< actionT >::attach (
    const std::shared_ptr< otherT > & other) [inline]
```

Attaches another execution, so that this one triggers it.

The attached execution's pending actions are run by this one's worker, and stopping this one stops that one too.

The actuators are given other's own actions, so the attachment is a pointer into other. What makes that safe is the record left on the other side: other is told which actuators hold it, and `~execution()` takes itself back out of them. The attacher's lifetime token guards those pointers, so an attached execution outliving its attacher follows nothing.

Parameters

other	- A <code>std::shared_ptr</code> owning the execution to trigger. Requiring it here is what keeps an execution that no <code>std::shared_ptr</code> owns from being attached at all, exactly as <code>bind_action_and_method()</code> requires one to bind. Attaching the same execution twice adds it twice; a single <code>detach()</code> still removes it completely, because <code>untangle::actuator::remove()</code> erases every match.
-------	---

Exceptions

<code>invalid_attachment</code>	- if other is attached already. An execution has at most one attacher, which is what keeps the attachment graph a forest and the cycle check below a walk rather than a search.
<code>invalid_attachment</code>	- if the attachment would close a cycle, which includes attaching an execution to itself. Driving a cycle recurses until the stack is gone, so it is refused here, where the caller still has a stack to be told on.

8.2.3.9 detach()

```
template<typename actionT>
template<typename otherT>
bool untangle::async::execution< actionT >::detach (
    otherT & other)    [inline]
```

Detaches an execution previously attached by [attach\(\)](#).

Both paths are unwired, the triggering one and the stopping one: an attachment that could only be half undone would leave this execution still able to stop one it no longer drives.

Parameters

other	- The execution to stop triggering. Detaching one that was never attached is not an error: the caller asked for a state that already holds.
-------	---

Returns

true - other was attached and is no longer.
false - other was not attached to this execution.

8.2.3.10 results()

```
template<typename actionT>
auto untangle::async::execution< actionT >::results () const    [inline]
```

The return values of the actions run by [run\(\)](#), in the order they ran.

Read it from `on_finished`, which fires once the queue has drained and before the execution reports itself finished, so the results are complete by the time the callback can see them.

Returned by value, under results `__mutex__` - the worker appends to the vector as it goes, so handing out a reference would hand out something being written.

Remarks

Only [run\(\)](#) fills this. The continuous worker started by [start\(\)](#) does not: it has no point at which a run is over, so there is nothing to hand back and nowhere to clear, and filling it would grow without bound. Deferred until there is a use case that wants it.

Returns

The results of the most recent run, or empty if none has produced any.

The documentation for this class was generated from the following file:

- `async.hpp`

8.3 untangle::async::execution_poll Class Reference

Execution poll class.

```
#include <async.hpp>
```

Public Member Functions

- `template<typename asyncexecT>`
`void add (asyncexecT &async_exec)`
Adds an [execution](#) object to the poll.
- `template<typename asyncexecT>`
`void remove (asyncexecT &async_exec)`
Removes an [execution](#) object from the poll.
- `auto is_running ()`
Checks if the execution objects are running.

Static Public Member Functions

- `static execution_poll & get ()`
Gets the single-tone instance.

8.3.1 Detailed Description

Execution poll class.

A single-tone class that may be used to verify if the polled [execution](#) objects have finished their processing. If at least one [execution](#) object is running then this poll has the "running" state.

Remarks

An [execution](#) object runs in a detached thread, hence it is not joinable. The caller can wait for an [execution](#) to be finished only by checking the "running" state([execution::is_running\(\)](#)). This class represents a convenient way to check any number of [execution](#) objects.

8.3.2 Member Function Documentation

8.3.2.1 add()

```
template<typename asyncexecT>
void untangle::async::execution_poll::add (
    asyncexecT & async_exec) [inline]
```

Adds an [execution](#) object to the poll.

Parameters

async_exec	An execution object.
------------	--------------------------------------

8.3.2.2 remove()

```
template<typename asyncexecT>
void untangle::async::execution_poll::remove (
    asyncexecT & async_exec) [inline]
```

Removes an [execution](#) object from the poll.

[add\(\)](#) stores a pointer to the object's `action_is_running`, so an execution destroyed while still registered leaves the poll calling into freed memory. `~execution()` calls this.

Parameters

async_exec	An execution object.
------------	--------------------------------------

8.3.2.3 is_running()

```
auto untangle::async::execution_poll::is_running () [inline]
```

Checks if the execution objects are running.

Returns

true - if at least one [execution](#) object in this poll is running.

8.3.2.4 get()

```
execution_poll & untangle::async::execution_poll::get () [inline], [static]
```

Gets the single-tone instance.

Returns

[execution_poll](#) An [execution_poll](#) instance.

The documentation for this class was generated from the following file:

- `async.hpp`

8.4 untangle::async::invalid_attachment Struct Reference

Invalid attachment exception.

```
#include <async.hpp>
```

Public Member Functions

- [invalid_attachment](#) (std::string text)
Construct a new invalid attachment object.
- const char * [what](#) () const noexcept override
The message text describing the reason of this exception.

8.4.1 Detailed Description

Invalid attachment exception.

Remarks

Raised by [execution::attach\(\)](#) when the attachment asked for would close a cycle - either between two executions that each attach the other, or an execution attached to itself. Driving such a cycle recurses until the stack is gone, so it is refused where the caller can still be told about it.

Deliberately not an `untangle::invalid_action`. That one reports a binding whose target has died, and `execution::execute_actions()` swallows it by design; a cycle is a caller error and must not be swallowed.

8.4.2 Constructor & Destructor Documentation

8.4.2.1 invalid_attachment()

```
untangle::async::invalid_attachment::invalid_attachment (
    std::string text) [inline], [explicit]
```

Construct a new invalid attachment object.

Parameters

text	- A message text, describing the reason of this exception.
------	--

8.4.3 Member Function Documentation

8.4.3.1 what()

```
const char * untangle::async::invalid_attachment::what () const [inline], [override], [noexcept]
```

The message text describing the reason of this exception.

Returns

const char* - The message text.

The documentation for this struct was generated from the following file:

- `async.hpp`

Chapter 9

File Documentation

9.1 async.hpp

```
00001 // Copyright (c) 2018 Nicolae Popescu. MIT License.
00002
00006 #pragma once
00007
00008 #include <actuator/actuator.hpp>
00009 #include <algorithm>
00010 #include <atomic>
00011 #include <condition_variable>
00012 #include <exception>
00013 #include <memory>
00014 #include <mutex>
00015 #include <print>
00016 #include <string>
00017 #include <thread>
00018 #include <vector>
00019
00020 namespace untangle {
00021 template <typename actionT>
00022 struct actuator;
00023
00024 namespace async {
00025 template <typename actionT>
00026 class execution;
00027 }
00028 } // namespace untangle
00029
00030 namespace untangle {
00031 namespace async {
00032
00035 struct invalid_attachment : std::exception {
00054     explicit invalid_attachment(std::string text) : message_(std::move(text)) {}
00055
00061     const char* what() const noexcept override { return message_.c_str(); }
00062
00063 private:
00064     std::string message_;
00065 };
00066
00079 struct attachment {
00081     std::weak_ptr<attachment> attacher;
00082 };
00083
00098 class execution_poll {
00099 public:
00105     template <typename asyncexecT>
00106     void add(asyncexecT& async_exec) {
00107         std::lock_guard<std::mutex> lock(actuator_mutex_);
00108
00109         if (!actuator_is_running_.is_connected()) {
00110             actuator_is_running_ = untangle::connect(async_exec.action_is_running);
00111         } else {
00112             actuator_is_running_.add(&async_exec.action_is_running);
00113         }
00114     }
00115
00124     template <typename asyncexecT>
00125     void remove(asyncexecT& async_exec) {
```

```

00126     std::lock_guard<std::mutex> lock(actuator_mutex_);
00127
00128     actuator_is_running_.remove(&async_exec.action_is_running);
00129 }
00130
00136 auto is_running() {
00137     std::lock_guard<std::mutex> lock(actuator_mutex_);
00138     actuator_is_running_();
00139
00140     auto result = false;
00141     for (const auto& ret : actuator_is_running_.results) {
00142         result |= ret;
00143     }
00144     return result;
00145 }
00146
00152 static execution_poll& get() {
00153     static execution_poll instance;
00154     return instance;
00155 }
00156
00157 private:
00158     execution_poll() = default;
00159     ~execution_poll() = default;
00160     actuator<std::function<bool(void)>> actuator_is_running_;
00161     mutable std::mutex actuator_mutex_;
00162 };
00163
00177 template <typename actionT>
00178 class execution {
00179     // An execution may attach one of a different specialisation - the smoke test attaches an
00180     // execution<function<void(int)>> to an execution<function<void(void)>> - and that is a different
00181     // class with no access to this one's members. attach() and detach() need to read and write
00182     // attachment_lifetime_ on it.
00183     template <typename otherActionT>
00184     friend class execution;
00185
00186 public:
00203     template <typename... Args>
00204     static std::shared_ptr<execution> create_instance(Args&&... args) {
00205         return std::make_shared<execution>(std::forward<Args>(args)...);
00206     }
00207
00216     execution() : execution(std::string(default_name)) {}
00217
00227     explicit execution(std::string exec_name) : name(std::move(exec_name)) {
00228         other_this_ = this;
00229         action_execute = untangle::bind(other_this_, &execution<actionT>::execute_actions);
00230         action_stop = untangle::bind(other_this_, &execution<actionT>::stop);
00231         action_is_running = untangle::bind(other_this_, &execution<actionT>::is_running);
00232     }
00233
00248     ~execution() {
00249         // Out of the attacher first, before the worker is even asked to stop: from here on nothing
00250         // may reach this object, and the actions about to be destroyed are the ones it holds.
00251         if (!attachment_lifetime_->attacher.expired()) {
00252             attacher_execute_->remove(&action_execute);
00253             attacher_stop_->remove(&action_stop);
00254         }
00255
00256         {
00257             std::lock_guard<std::mutex> lock(action_mutex_);
00258             started_ = false;
00259             stopped_ = true;
00260         }
00261
00262         action_cv_.notify_all();
00263
00264         while (running_.load()) { // time of check
00265             std::this_thread::sleep_for(std::chrono::milliseconds(1));
00266         }
00267
00268         // time of use: this object is freed once the destructor returns, so the check above is only
00269         // safe because running_ is the last thing the worker touches
00270         execution_poll::get().remove(*this);
00271     }
00272
00293     template <typename classT, typename T>
00294     static void bind_action_and_method(actionT& action, const std::shared_ptr<classT>& obj,
00295                                       T classT::* method,
00296                                       const std::shared_ptr<execution>& async_exec) {
00297         actionT async_action = untangle::bind(obj, method);
00298
00299         action = [wp = std::weak_ptr<execution>(async_exec),
00300                 async_action](auto... args) -> actionT::result_type {
00301             // lock() also keeps the execution alive for the duration of the call
00302             const auto exec = wp.lock();

```

```

00303     if (!exec) {
00304         throw invalid_action("bind: invalid execution");
00305     }
00306     exec->add_action(async_action, args...);
00307     return typename actionT::result_type();
00308 };
00309 }
00310
00322 template <typename T>
00323 static void bind_action_and_function(actionT& action, const T& Fn,
00324                                     const std::shared_ptr<execution>& async_exec) {
00325     actionT async_action = Fn;
00326
00327     action = [wp = std::weak_ptr<execution>(async_exec),
00328             async_action](auto... args) -> actionT::result_type {
00329         const auto exec = wp.lock();
00330         if (!exec) {
00331             throw invalid_action("bind: invalid execution");
00332         }
00333         exec->add_action(async_action, args...);
00334         return typename actionT::result_type();
00335     };
00336 }
00337
00353 bool is_running() const { return running_.load() && !finishing_.load(); }
00354
00375 bool is_busy() const {
00376     std::lock_guard<std::mutex> lock(action_mutex_);
00377     return !action_list_.empty() || executing_action_.load();
00378 }
00379
00380 void run() {
00381     running_ = true;
00382     finishing_ = false;
00383     collecting_results_ = true;
00384     this_thread_ = std::thread(&execution::execute, this);
00385     this_thread_.detach();
00386 }
00387
00388 void start() {
00389     running_ = true;
00390     finishing_ = false;
00391     collecting_results_ = false;
00392     {
00393         std::lock_guard<std::mutex> lock(action_mutex_);
00394         started_ = true;
00395         stopped_ = false;
00396     }
00397
00398     this_thread_ = std::thread(&execution::loop, this);
00399     this_thread_.detach();
00400 }
00401
00408 void stop() {
00409     {
00410         std::lock_guard<std::mutex> lock(action_mutex_);
00411         started_ = false;
00412         stopped_ = true;
00413     }
00414
00415     // The worker drains what is already queued before it leaves loop(), so stop() does not have to
00416     // spin on the list; waking it is enough.
00417     action_cv_.notify_all();
00418
00419     if (actuator_stop_.is_connected()) {
00420         actuator_stop_();
00421     }
00422 }
00423
00431 template <typename... Args>
00432 void add_action(actionT action, Args... args) {
00433     {
00434         std::lock_guard<std::mutex> lock(action_mutex_);
00435
00436         // Once stopped, the worker is on its way out and would never reach this action; dropping it
00437         // here is what keeps it from sitting in the list looking as though it were pending.
00438         if (stopped_) {
00439             std::println(stderr, "warning: execution '{}{}' is stopped, action not added", name);
00440             return;
00441         }
00442
00443         action_list_.push_back(std::bind(action, args...));
00444     }
00445
00446     action_cv_.notify_one();
00447 }
00448

```

```

00474 template <typename otherT>
00475 void attach(const std::shared_ptr<otherT>& other) {
00476     if (static_cast<const void*>(other.get()) == static_cast<const void*>(this)) {
00477         throw invalid_attachment("attach: an execution cannot be attached to itself");
00478     }
00479
00480     if (!other->attachment_lifetime_->attacher.expired()) {
00481         throw invalid_attachment("attach: that execution is attached already");
00482     }
00483
00484     // Refusing an execution that has an attacher already leaves every execution with at most one,
00485     // so the graph is a forest and this new edge closes a cycle exactly when other is somewhere up
00486     // this execution's own chain of attachers. Walking it is the whole check.
00487     for (auto link = attachment_lifetime_->attacher.lock(); link; link = link->attacher.lock()) {
00488         if (link == other->attachment_lifetime_) {
00489             throw invalid_attachment("attach: the attachment would close a cycle");
00490         }
00491     }
00492
00493     if (!actuator_execute_.is_connected()) {
00494         actuator_execute_ = untangle::connect(other->action_execute);
00495     } else {
00496         actuator_execute_.add(&other->action_execute);
00497     }
00498
00499     if (!actuator_stop_.is_connected()) {
00500         actuator_stop_ = untangle::connect(other->action_stop);
00501     } else {
00502         actuator_stop_.add(&other->action_stop);
00503     }
00504
00505     // What ~execution() needs to take itself back out of these actuators: the link proves this
00506     // attacher is still there, and the two pointers are only ever followed while it does.
00507     other->attachment_lifetime_->attacher = attachment_lifetime_;
00508     other->attacher_execute_ = &actuator_execute_;
00509     other->attacher_stop_ = &actuator_stop_;
00510 }
00511
00524 template <typename otherT>
00525 bool detach(otherT& other) {
00526     // The actuator's own action list is the record of what is attached; nothing else has to keep
00527     // one. An action is stored as a pointer, so the attachment is found by identity.
00528     const auto& actions = actuator_execute_.actions;
00529     if (std::find(actions.begin(), actions.end(), &other.action_execute) == actions.end()) {
00530         return false;
00531     }
00532
00533     actuator_execute_.remove(&other.action_execute);
00534     actuator_stop_.remove(&other.action_stop);
00535
00536     other.attachment_lifetime_->attacher.reset();
00537     other.attacher_execute_ = nullptr;
00538     other.attacher_stop_ = nullptr;
00539
00540     return true;
00541 }
00542
00558 auto results() const {
00559     std::lock_guard<std::mutex> lock(results_mutex_);
00560     return results_;
00561 }
00562
00563 static constexpr auto default_name = "default_name";
00564
00565 std::function<void(void)> action_execute;
00566 std::function<void(void)> action_stop;
00567 std::function<bool(void)> action_is_running;
00568 std::function<void(void)> on_finished;
00569 std::string name = default_name;
00570
00571 private:
00572 using queued_action_t = std::function<typename actionT::result_type(void)>;
00573
00582 void execute_action(queued_action_t& action) {
00583     if constexpr (std::is_void_v<typename actionT::result_type>) {
00584         action();
00585     } else {
00586         auto value = action();
00587
00588         if (collecting_results_) {
00589             std::lock_guard<std::mutex> lock(results_mutex_);
00590             results_.push_back(std::move(value));
00591         }
00592     }
00593 }
00594
00604 std::size_t execute_actions() {

```

```

00605     std::size_t actions_run = 0;
00606
00607     // The action is taken off the list under the lock and invoked with the lock released: an action
00608     // is caller code that may run for a while, and may itself call add_action().
00609     for (;;) {
00610         queued_action_t action;
00611
00612         {
00613             std::lock_guard<std::mutex> lock(action_mutex_);
00614             if (action_list_.empty()) {
00615                 break;
00616             }
00617
00618             action = std::move(action_list_.front());
00619             action_list_.pop_front();
00620
00621             // Set here, under the lock that emptied the list, so that there is no instant in which the
00622             // list reads empty while this action has not yet run. is_busy() takes the same lock, so it
00623             // sees the pop and this together or neither.
00624             executing_action_ = true;
00625         }
00626
00627         // An action bound to an object that has since died throws invalid_action. Letting it leave a
00628         // thread function calls std::terminate, and a detached worker gives no one a chance to catch
00629         // it; the actuator drops such an action, and so does this.
00630         try {
00631             execute_action(action);
00632         } catch (const invalid_action& ia) {
00633             std::println(stderr, "warning: execution '{}' dropped an invalid action: {}", name,
00634                 ia.what());
00635         }
00636
00637         executing_action_ = false;
00638
00639         // Counted whether or not it reported a dead binding: it came off the queue and the queue is
00640         // what the notification is about.
00641         ++actions_run;
00642     }
00643
00644     if (actuator_execute_.is_connected()) {
00645         actuator_execute_();
00646     }
00647
00648     return actions_run;
00649 }
00650
00651 void notify_finished(std::size_t actions_run) {
00652     if (actions_run == 0 || !on_finished) {
00653         return;
00654     }
00655
00656     {
00657         // An action may queue another, so a pass that drained can leave more behind it. That is the
00658         // next batch, not the end of this one.
00659         std::lock_guard<std::mutex> lock(action_mutex_);
00660         if (!action_list_.empty()) {
00661             return;
00662         }
00663     }
00664
00665     on_finished();
00666 }
00667
00668 void execute() {
00669     // The results belong to this run: an execution that is run twice reports the second run's
00670     // results, not both runs' appended together.
00671     {
00672         std::lock_guard<std::mutex> lock(results_mutex_);
00673         results_.clear();
00674     }
00675
00676     const auto actions_run = execute_actions();
00677
00678     // Set before the callback, not after: the run is over by the time it is told so, and a callback
00679     // that asks is_running() has to be told the truth. `running_` cannot be cleared here instead -
00680     // ~execution() waits on it and may free this object the moment it reads false, so it has to
00681     // stay the last thing this worker touches.
00682     finishing_ = true;
00683
00684     // Before running_ is cleared, so a caller waiting on the poll cannot see the execution finish
00685     // and read the results before this has filled them.
00686     notify_finished(actions_run);
00687
00688     std::cout << "finishing_thread" << std::endl;
00689
00690     running_ = false;
00691 }

```

```

00698
00699 void loop() {
00700     for (;;) {
00701     {
00702         std::unique_lock<std::mutex> lock(action_mutex_);
00703
00704         // A bounded wait rather than a plain one: an attached execution has its own list and no way
00705         // to notify this condition variable, so the worker still has to look in on it periodically.
00706         action_cv_.wait_for(lock, std::chrono::milliseconds(10),
00707             [this] { return !action_list_.empty() || !started_; });
00708
00709         if (!started_ && action_list_.empty()) {
00710             break;
00711         }
00712     }
00713
00714     notify_finished(execute_actions());
00715 }
00716
00717 // What was queued before stop() still belongs to this execution; nothing can have been added
00718 // after it, because add_action() refuses once stopped. A batch is a batch whichever side of the
00719 // stop it drained on, so it is reported like any other; a stop with nothing left to run reports
00720 // nothing, because nothing finished.
00721 notify_finished(execute_actions());
00722
00723 std::cout << "thread finished" << std::endl;
00724
00725 // Must stay last: ~execution() may free this object the moment it reads false.
00726 running_ = false;
00727 }
00728
00729 std::thread this_thread_;
00730
00731 std::atomic_bool started_ = {false};
00732 std::atomic_bool running_ = {false};
00733
00734 // Set by stop(), and the difference between "not started yet" and "finished for good": actions
00735 // may be queued before run()/start(), but not after stop().
00736 std::atomic_bool stopped_ = {false};
00737
00738 // action_list_, started_ and stopped_ are written by every thread that calls add_action() or
00739 // stop() and read by the worker; nothing touches them outside this mutex. action_cv_ is what
00740 // replaced the
00741 // spin in loop() and in stop().
00742 mutable std::mutex action_mutex_;
00743 std::condition_variable action_cv_;
00744
00745 std::list<std::function<typename actionT::result_type(void)>> action_list_;
00746
00755 using void_actuator = actuator<std::function<void(void)>>;
00756
00757 void_actuator actuator_execute_;
00758 void_actuator actuator_stop_;
00759
00773 std::shared_ptr<attachment> attachment_lifetime_ = std::make_shared<attachment>();
00774
00788 void_actuator* attacher_execute_ = nullptr;
00789 void_actuator* attacher_stop_ = nullptr;
00790
00791 // std::vector cannot hold void type; use an arbitrary type e.g. int
00792 using resultT = std::conditional<std::is_void<typename actionT::result_type>::value, int,
00793     typename actionT::result_type>;
00794
00803 std::vector<typename resultT::type> results_;
00804
00811 mutable std::mutex results_mutex_;
00812
00820 std::atomic_bool collecting_results_ = {false};
00821
00830 std::atomic_bool finishing_ = {false};
00831
00840 std::atomic_bool executing_action_ = {false};
00841
00842 execution* other_this_;
00843 };
00844 } // namespace async
00845 } // namespace untangle

```

Index

- ~execution
 - untangle::async::execution< actionT >, [19](#)
- add
 - untangle::async::execution_poll, [25](#)
- add_action
 - untangle::async::execution< actionT >, [22](#)
- async, [1](#)
- attach
 - untangle::async::execution< actionT >, [23](#)
- bind_action_and_function
 - untangle::async::execution< actionT >, [21](#)
- bind_action_and_method
 - untangle::async::execution< actionT >, [20](#)
- create_instance
 - untangle::async::execution< actionT >, [20](#)
- detach
 - untangle::async::execution< actionT >, [23](#)
- execution
 - untangle::async::execution< actionT >, [19](#)
- get
 - untangle::async::execution_poll, [26](#)
- invalid_attachment
 - untangle::async::invalid_attachment, [27](#)
- is_busy
 - untangle::async::execution< actionT >, [21](#)
- is_running
 - untangle::async::execution< actionT >, [21](#)
 - untangle::async::execution_poll, [26](#)
- namespace untangle: functions, [13](#)
- remove
 - untangle::async::execution_poll, [26](#)
- results
 - untangle::async::execution< actionT >, [24](#)
- stop
 - untangle::async::execution< actionT >, [22](#)
- untangle, [15](#)
- untangle::async::attachment, [17](#)
- untangle::async::execution< actionT >, [17](#)
 - ~execution, [19](#)
 - add_action, [22](#)
 - attach, [23](#)
 - bind_action_and_function, [21](#)
 - bind_action_and_method, [20](#)
 - create_instance, [20](#)
 - detach, [23](#)
 - execution, [19](#)
 - is_busy, [21](#)
 - is_running, [21](#)
 - results, [24](#)
 - stop, [22](#)
- untangle::async::execution_poll, [25](#)
 - add, [25](#)
 - get, [26](#)
 - is_running, [26](#)
 - remove, [26](#)
- untangle::async::invalid_attachment, [27](#)
 - invalid_attachment, [27](#)
 - what, [27](#)
- what
 - untangle::async::invalid_attachment, [27](#)